SOFTWARE REQUIREMENTS SPECIFICATION

AI-Powered Analytics Intelligence Platform

GA4 · Google Search Console · Google Ads · Facebook Ads

MERN Stack | Vite + React | Tailwind CSS v3 | Anthropic Claude AI

VERSION 1.0 — FINAL (Includes Super Admin Panel)

Field	Value
Document Version	1.0 – Final (Includes Super Admin Panel)
Status	FINAL – Ready for Cursor Agent Build
Stack	MongoDB · Express.js · React (Vite) · Node.js
Styling	Tailwind CSS v3
AI Model	Anthropic Claude 3.5 Sonnet (primary) | GPT-4o (fallback)
API Integrations	GA4 · Google Search Console · Google Ads · Facebook Ads (Meta)
Build Method	Cursor AI Agent – Zero Human Intervention
Phase	Phase 1 – All 4 Integrations
Product Type	Market SaaS – Web Application
Database	MongoDB Atlas (Cloud)
Date	March 2, 2026

1. Document Purpose & Agentic Build Instructions

This SRS is the single source of truth for a Cursor AI Agent operating in fully autonomous mode with zero human intervention. All four API integrations — Google Analytics 4, Google Search Console, Google Ads, and Facebook Ads — are Phase 1 requirements and must all be built together. The agent must read this document end-to-end before writing any code.

> ⚙️ **AGENT: Read the entire document before starting. Do not ask for clarification. Do not pause for human review. All 4 API integrations are mandatory in Phase 1. If any decision is not covered here, apply the most secure, production-grade MERN default.**

1.1 Core Build Rules (Non-Negotiable)

- MERN stack only — MongoDB, Express.js, React (Vite), Node.js — no other framework
- Tailwind CSS v3 for all styling — no CSS-in-JS, no Styled Components, no plain CSS files
- All 4 API integrations built in Phase 1 — GA4, GSC, Google Ads, Facebook Ads
- Anthropic Claude 3.5 Sonnet as primary AI — fallback to OpenAI GPT-4o only if Claude API fails
- NO hard-coded analytics rules — all data interpretation done by AI prompt engine at runtime
- NO static thresholds or if/else logic that interprets what analytics data means
- All prompt templates stored in /server/prompts/*.txt — never embedded in code
- All secrets in .env files — never committed to git, never hard-coded
- Full GDPR compliance by design — no personal visitor data stored

2. System Overview

The AI-Powered Analytics Intelligence Platform is a multi-tenant SaaS web application. It connects to all four data sources — Google Analytics 4 (GA4), Google Search Console (GSC), Google Ads API, and Facebook Ads (Meta Marketing API) — fetches aggregated performance data, and uses Anthropic Claude AI with dynamic prompt engineering to convert raw cross-channel metrics into clear, human-readable insights in one unified dashboard.

2.1 Architecture Overview

Layer	Technology	Responsibility
Frontend	React 18 (Vite) + React Router v6 + Tailwind CSS v3	All UI: dashboard, charts, AI chat, auth, account connection pages
State Management	Zustand	Global: user, connected accounts, date range, active data source
HTTP Client	Axios with interceptors	API calls, JWT attachment, auto token refresh on 401
Backend	Node.js + Express.js (REST API)	Business logic, all 4 API orchestrations, AI

Layer	Technology	Responsibility
		engine
Auth – Google	Passport.js + passport-google-oauth20 + JWT	Google OAuth for GA4, GSC, Google Ads token management
Auth – Facebook	passport-facebook + JWT	Facebook OAuth for Meta Ads token management
Database	MongoDB Atlas + Mongoose ODM	Users, accounts, tokens, cache, AI logs
Cache	node-cache (in-memory)	All 4 API response caching — 30 min TTL each
GA4 Integration	googleapis (google-analytics-data v1beta)	Website traffic, sessions, users, bounce rate, conversions
GSC Integration	googleapis (webmasters v3)	Search clicks, impressions, CTR, average position, top queries
Google Ads	google-ads-api npm package	Campaigns, ad groups, keywords, impressions, spend, ROAS, CPC
Facebook Ads	facebook-nodejs-business-sdk (official Meta SDK)	Campaigns, adsets, ads, reach, spend, ROAS, CPM, CPC, CTR
AI Engine	@anthropic-ai/sdk	Generate insights from all 4 sources via dynamic prompt engineering
Email	Resend (resend.com)	Verification emails, password reset, transactional notifications
Hosting (Dev)	Vite dev server + nodemon	Local development — Vite proxies /api to Express on port 5000
Hosting (Prod)	Vercel/Netlify (client) + Railway/Render (server)	Production cloud deployment

2.2 System Data Flow

> ⚙️ **AGENT: Implement every step as a discrete service or function. No logic should be combined across steps.**

- Step 1 — User registers/logs in → JWT issued → stored in Zustand authStore
- Step 2 — User connects Google account via OAuth → GA4 + GSC + Google Ads tokens encrypted → saved to GoogleToken collection
- Step 3 — User connects Facebook account via OAuth → Meta Ads token encrypted → saved to FacebookToken collection
- Step 4 — User selects GA4 property, GSC site, Google Ads customer ID, Facebook Ad Account → saved to UserAccounts document
- Step 5 — Dashboard loads → React fires parallel requests to all 4 data endpoints → Express checks cache for each
- Step 6 — Cache miss on any source → Express calls that API with valid decrypted token → saves response to cache → returns to React
- Step 7 — User types AI question → React POSTs to /api/ai/ask with question + date range

- Step 8 — PromptBuilder fetches latest data from all 4 sources → assembles system prompt + full context + user question
- Step 9 — Express calls Anthropic Claude API → response saved to MongoDB → returned to React as markdown
- Step 10 — React renders AI response in chat panel using react-markdown with remark-gfm

3. MongoDB Schemas (Mongoose)

⚙️ **AGENT: Create all schemas in /server/models/. One file per model. Export each as default. Enable timestamps: true on all. Use mongoose.Schema.Types.ObjectId for all refs.**

3.1 User Schema — /server/models/User.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
email	String	Yes	Unique, lowercase, trimmed, indexed
passwordHash	String	No	Null for social-only accounts. bcrypt 12 rounds
displayName	String	Yes	From OAuth profile or registration form
avatar	String	No	Profile photo URL from Google
emailVerified	Boolean	Yes	Default false. True after email confirmation
emailVerifyToken	String	No	UUID. Null after verification
passwordResetToken	String	No	UUID. Null after use
passwordResetExp	Date	No	Reset token expiry — now + 1 hour
googleId	String	No	Unique Google account ID. Sparse index
facebookId	String	No	Unique Facebook account ID. Sparse index
loginAttempts	Number	Yes	Default 0. Increment on failed login
lockUntil	Date	No	Account locked until this timestamp
createdAt	Date	auto	Mongoose timestamps
updatedAt	Date	auto	Mongoose timestamps

3.2 GoogleToken Schema — /server/models/GoogleToken.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
userId	ObjectId	Yes	Ref: User. Unique. One doc per user
accessToken	String	Yes	AES-256-GCM encrypted

Field	Type	Required	Notes
refreshToken	String	Yes	AES-256-GCM encrypted
expiresAt	Date	Yes	Access token expiry timestamp
scope	String	Yes	Space-separated OAuth scopes granted
createdAt	Date	auto	Mongoose timestamps
updatedAt	Date	auto	Mongoose timestamps

3.3 FacebookToken Schema — /server/models/FacebookToken.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
userId	ObjectId	Yes	Ref: User. Unique. One doc per user
accessToken	String	Yes	AES-256-GCM encrypted long-lived user token
expiresAt	Date	Yes	Facebook long-lived tokens expire in ~60 days
facebookUserId	String	Yes	Facebook User ID — used to list ad accounts
scope	String	Yes	Comma-separated granted permissions
createdAt	Date	auto	Mongoose timestamps
updatedAt	Date	auto	Mongoose timestamps

⚙️ AGENT: Facebook long-lived tokens expire in ~60 days. Implement a scheduled job (node-cron) that runs daily, checks FacebookToken.expiresAt, and sends a re-authentication reminder email 7 days before expiry. Automatically attempt token refresh using the Graph API /oauth/access_token endpoint with grant_type=fb_exchange_token.

3.4 UserAccounts Schema — /server/models/UserAccounts.js

Stores the user's selected accounts for all 4 data sources. One document per user.

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
userId	ObjectId	Yes	Ref: User. Unique — one doc per user
ga4PropertyId	String	No	e.g. properties/123456789
ga4PropertyName	String	No	Human-readable GA4 property name
ga4AccountId	String	No	Parent GA4 account ID
gscSiteUrl	String	No	e.g. https://example.com/ or sc-domain:example.com
gscPermission	String	No	siteOwner | siteFullUser | siteRestrictedUser
googleAdsCustomerId	String	No	10-digit Google Ads customer ID — no dashes
googleAdsAccountName	String	No	Human-readable Google Ads account name

Field	Type	Required	Notes
googleAdsCurrencyCode	String	No	Account currency e.g. USD, GBP, INR
facebookAdAccountId	String	No	Meta ad account ID e.g. act_123456789
facebookAdAccountNam e	String	No	Human-readable Facebook ad account name
facebookAdCurrency	String	No	Ad account currency e.g. USD
createdAt	Date	auto	Mongoose timestamps
updatedAt	Date	auto	Mongoose timestamps

⚙️ **AGENT: All account fields are optional. A user may connect some sources but not others. The frontend must show independent connection status badges for each of the 4 sources. Each source can be connected or disconnected independently without affecting others.**

3.5 AnalyticsCache Schema — /server/models/AnalyticsCache.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
userId	ObjectId	Yes	Ref: User. Compound index with source + reportType + dates
source	String	Yes	Enum: ga4 | gsc | google-ads | facebook-ads
reportType	String	Yes	e.g. overview | campaigns | queries | traffic — varies per source
accountId	String	Yes	GA4 propertyId, GSC siteUrl, Google Ads customerId, or FB adAccountId
dateRangeStart	String	Yes	YYYY-MM-DD
dateRangeEnd	String	Yes	YYYY-MM-DD
data	Mixed	Yes	Raw API response object
fetchedAt	Date	Yes	When fetched from the external API
expiresAt	Date	Yes	fetchedAt + 30 minutes. TTL index on this field

⚙️ **AGENT: Create a MongoDB TTL index: AnalyticsCacheSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 }). MongoDB auto-deletes expired documents. This applies to cache from all 4 sources.**

3.6 Conversation Schema — /server/models/Conversation.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
userId	ObjectId	Yes	Ref: User. Indexed
title	String	No	Auto-generated from first question (first 60 chars)

Field	Type	Required	Notes
sources	[String]	No	Which data sources were active: ['ga4','gsc','google-ads','facebook-ads']
createdAt	Date	auto	Mongoose timestamps
updatedAt	Date	auto	Mongoose timestamps

3.7 Message Schema — /server/models/Message.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
conversationId	ObjectId	Yes	Ref: Conversation. Indexed
role	String	Yes	Enum: user | assistant
content	String	Yes	Markdown string
model	String	No	AI model used e.g. claude-3-5-sonnet-20241022
tokensUsed	Number	No	Total tokens (prompt + completion)
latencyMs	Number	No	Time from request to response in ms
createdAt	Date	auto	Mongoose timestamps

3.8 WeeklyInsight Schema — /server/models/WeeklyInsight.js

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
userId	ObjectId	Yes	Ref: User. Indexed
content	String	Yes	AI-generated markdown insight covering all active sources
sources	[String]	Yes	Which sources were used: ['ga4','gsc','google-ads','facebook-ads']
expiresAt	Date	Yes	createdAt + 24 hours. TTL index on this field
createdAt	Date	auto	Mongoose timestamps

4. Project Folder Structure

⚙️ **AGENT: Scaffold this exact structure before writing any logic. Monorepo with /client and /server directories. Do NOT use Next.js. Do NOT mix frontend and backend files.**

```
/ (monorepo root)
├── client/                          # React + Vite frontend
```

```
│   ├── public/
│   ├── src/
│   │   ├── api/                      # Axios call functions
│   │   │   ├── authApi.js
│   │   │   ├── ga4Api.js
│   │   │   ├── gscApi.js
│   │   │   ├── googleAdsApi.js
│   │   │   ├── facebookAdsApi.js
│   │   │   └── aiApi.js
│   │   ├── components/
│   │   │   ├── ui/                   # Button, Input, Modal, Badge, Spinner, Skeleton
│   │   │   ├── charts/               # Recharts wrappers: Line, Bar, Donut, Area
│   │   │   ├── dashboard/            # KpiCard, DataTable, DateRangePicker
│   │   │   ├── connections/          # ConnectGoogleCard, ConnectFacebookCard,
AccountSelector
│   │   │   └── ai/                   # ChatPanel, MessageBubble, SuggestedQuestions
│   │   ├── hooks/
│   │   │   ├── useGA4.js
│   │   │   ├── useGSC.js
│   │   │   ├── useGoogleAds.js
│   │   │   ├── useFacebookAds.js
│   │   │   └── useAuth.js
│   │   ├── pages/
│   │   │   ├── LoginPage.jsx
│   │   │   ├── RegisterPage.jsx
│   │   │   ├── DashboardPage.jsx
│   │   │   ├── ConnectAccountsPage.jsx  # Step: connect all 4 sources
│   │   │   ├── SelectAccountsPage.jsx   # Step: pick specific accounts
│   │   │   └── SettingsPage.jsx
│   │   ├── store/                    # Zustand stores
│   │   │   ├── authStore.js
│   │   │   ├── accountsStore.js        # All 4 selected accounts
│   │   │   └── dateRangeStore.js
│   │   ├── utils/                    # formatters, date helpers, currency
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css                 # Tailwind directives only
│   ├── index.html
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── vercel.json                   # For Vercel deployment
│   └── netlify.toml                  # For Netlify deployment
│
├── server/                           # Express.js backend
│   ├── config/
│   │   ├── db.js                     # Mongoose connection to Atlas
│   │   ├── passport.js               # Google + Facebook OAuth strategies
│   │   └── constants.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── ga4Controller.js
│   │   ├── gscController.js
```

```
│   │   ├── googleAdsController.js
│   │   ├── facebookAdsController.js
│   │   └── aiController.js
│   ├── middleware/
│   │   ├── auth.js                    # JWT verification
│   │   ├── rateLimiter.js
│   │   └── errorHandler.js
│   ├── models/                        # All 8 Mongoose models (Section 3)
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── accountRoutes.js
│   │   ├── ga4Routes.js
│   │   ├── gscRoutes.js
│   │   ├── googleAdsRoutes.js
│   │   ├── facebookAdsRoutes.js
│   │   └── aiRoutes.js
│   ├── services/
│   │   ├── googleAuthService.js       # Token encrypt/decrypt/refresh for Google
│   │   ├── facebookAuthService.js     # Token encrypt/decrypt/refresh for Facebook
│   │   ├── ga4Service.js              # GA4 API calls + cache
│   │   ├── gscService.js              # GSC API calls + cache
│   │   ├── googleAdsService.js        # Google Ads API calls + cache
│   │   ├── facebookAdsService.js      # Facebook Ads API calls + cache
│   │   ├── promptBuilder.js           # Assembles prompts from all 4 sources
│   │   ├── claudeService.js           # Anthropic API client
│   │   ├── openaiService.js           # OpenAI fallback client
│   │   └── emailService.js
│   ├── prompts/
│   │   ├── system.txt
│   │   ├── weekly-insight.txt
│   │   └── suggested-questions.txt
│   ├── utils/
│   │   ├── encrypt.js                 # AES-256-GCM encrypt/decrypt
│   │   ├── catchAsync.js              # Async error wrapper for routes
│   │   └── dateHelpers.js
│   ├── app.js                         # Express app setup
│   └── server.js                      # HTTP listen entry point
│
├── .env                               # Local secrets (gitignored)
├── .env.example                       # All variable names, no real values
├── .gitignore
├── package.json                       # Root — concurrently dev script
└── README.md
```

5. REST API Endpoint Specification

All routes prefixed with /api. Protected routes require Authorization: Bearer <JWT>. All responses are JSON. Errors follow: { success: false, message: string, code?: string }.

5.1 Auth Routes — /api/auth

Method	Endpoint	Auth	Request Body	Response
POST	/register	No	{ name, email, password }	201 { message: 'Verification email sent' }
GET	/verify-email/:token	No	Token in URL	200 { message: 'Email verified' }
POST	/login	No	{ email, password }	200 { token, user }
POST	/logout	Yes	—	200 { message }
POST	/forgot-password	No	{ email }	200 { message } — always same response
POST	/reset-password	No	{ token, newPassword }	200 { message }
GET	/google	No	—	302 Redirect to Google OAuth
GET	/google/callback	No	?code=&state=	302 Redirect to client /auth/callback?token=JWT
GET	/facebook	No	—	302 Redirect to Facebook OAuth
GET	/facebook/callback	No	?code=&state=	302 Redirect to client /auth/callback?token=JWT
GET	/me	Yes	—	200 { user, connectedSources: ['ga4','gsc',...] }
DELETE	/me	Yes	—	200 { message } — cascade deletes all data

5.2 Account Connection Routes — /api/accounts

Method	Endpoint	Auth	Description	Response
GET	/ga4/list	Yes	List all GA4 properties from Google API	200 [{ id, name, accountId }]
GET	/gsc/list	Yes	List all GSC sites from Google API	200 [{ siteUrl, permissionLevel }]
GET	/google-ads/list	Yes	List all Google Ads accounts accessible by user	200 [{ customerId, name, currencyCode }]
GET	/facebook-ads/list	Yes	List all Facebook Ad Accounts for the user	200 [{ id, name, currency, status }]
POST	/select	Yes	Save all selected accounts to UserAccounts doc	200 { message, accounts }
GET	/active	Yes	Get user's currently active account selections	200 { ga4, gsc, googleAds, facebookAds }
DELETE	/disconnect/google	Yes	Delete GoogleToken + clear GA4/GSC/Ads selections	200 { message }

Method	Endpoint	Auth	Description	Response
DELETE	/disconnect/facebook	Yes	Delete FacebookToken + clear Facebook selections	200 { message }

5.3 GA4 Routes — /api/ga4

⚙️ **AGENT: All GA4 routes require query params: startDate (YYYY-MM-DD) and endDate (YYYY-MM-DD). Validate on every request. Check cache before calling Google API.**

Method	Endpoint	GA4 Dimensions	GA4 Metrics
GET	/overview	date	activeUsers, sessions, bounceRate, averageSessionDuration, screenPageViews
GET	/timeseries	date	activeUsers, sessions, screenPageViews (metric from query param)
GET	/traffic	sessionDefaultChannelGroup, sessionSource	sessions, activeUsers, bounceRate
GET	/pages	pagePath, pageTitle	screenPageViews, activeUsers, averageSessionDuration
GET	/devices	deviceCategory	sessions, activeUsers
GET	/geo	country	activeUsers, sessions
GET	/compare	date	activeUsers, sessions — current + prior period

5.4 GSC Routes — /api/gsc

Method	Endpoint	GSC Dimensions	GSC Metrics
GET	/overview	date	clicks, impressions, ctr, position
GET	/queries	query	clicks, impressions, ctr, position
GET	/pages	page	clicks, impressions, ctr, position
GET	/devices	device	clicks, impressions, ctr, position
GET	/countries	country	clicks, impressions, ctr, position
GET	/timeseries	date	clicks, impressions, ctr, position

5.5 Google Ads Routes — /api/google-ads

⚙️ **AGENT: Use the google-ads-api npm package. All requests require the user's googleAdsCustomerId from UserAccounts. Use GAQL (Google Ads Query Language) for all data fetches.**

Method	Endpoint	GAQL Resource	Metrics Fetched
GET	/overview	campaign	metrics.cost_micros, metrics.impressions, metrics.clicks, metrics.conversions, metrics.ctr, metrics.average_cpc
GET	/campaigns	campaign	campaign.id, campaign.name, campaign.status, all overview metrics — top 20 by spend
GET	/adgroups	ad_group	ad_group.id, ad_group.name, ad_group.status, metrics.cost_micros, metrics.clicks, metrics.impressions
GET	/keywords	keyword_view	keyword.text, keyword.match_type, metrics.cost_micros, metrics.clicks, metrics.impressions, metrics.average_cpc
GET	/timeseries	campaign	segments.date, metrics.cost_micros, metrics.clicks, metrics.impressions — aggregated daily
GET	/compare	campaign	All overview metrics for current and prior period

⚙️ AGENT: cost_micros is in micro-currency (divide by 1,000,000 for actual spend). Always convert before sending to frontend. Include currencyCode from UserAccounts.googleAdsCurrencyCode in every response.

5.6 Facebook Ads Routes — /api/facebook-ads

⚙️ AGENT: Use the official facebook-nodejs-business-sdk. All requests use the user's facebookAdAccountId from UserAccounts. The base URL is: https://graph.facebook.com/v19.0/

Method	Endpoint	FB API Endpoint	Fields / Metrics Fetched
GET	/overview	/{ad_account_id}/insights	spend, impressions, clicks, ctr, cpc, cpm, reach, frequency, actions (conversions), roas
GET	/campaigns	/{ad_account_id}/campaigns	id, name, status, objective — then insights for each: spend, impressions, clicks, ctr, roas
GET	/adsets	/{ad_account_id}/adsets	id, name, status, targeting — then insights: spend, impressions, clicks, ctr
GET	/ads	/{ad_account_id}/ads	id, name, status, creative — then insights: spend, impressions, clicks, ctr
GET	/timeseries	/{ad_account_id}/insights	date_start, date_stop (daily breakdown), spend, impressions, clicks, reach
GET	/compare	/{ad_account_id}/insights	All overview metrics for current and prior period using date_preset or time_range

⚙️ AGENT: Facebook Ads insights API uses time_range: { since: YYYY-MM-DD, until: YYYY-MM-DD }. The level param controls granularity: level=account | campaign | adset | ad. Always pass level explicitly. For ROAS, use the action_values field and divide by spend.

5.7 AI Routes — /api/ai

Method	Endpoint	Auth	Request / Params	Response
POST	/ask	Yes	{ question, conversationId?, dateRange, activeSources[] }	200 { answer (markdown), conversationId, messageId }
GET	/conversations	Yes	—	200 [{ _id, title, createdAt, messageCount }]
GET	/conversations/:id	Yes	—	200 { _id, title, messages: [{ role, content, createdAt }] }
DELETE	/conversations/:id	Yes	—	200 { message }
GET	/weekly-insight	Yes	—	200 { content (markdown), sources[], generatedAt, expiresAt }
POST	/weekly-insight/refresh	Yes	—	200 { content (markdown), sources[], generatedAt }
GET	/suggested-questions	Yes	—	200 { questions: [string] } — 4 to 6 questions

6. Authentication Implementation

6.1 Email/Password Auth

- bcrypt with saltRounds = 12 for all password hashing
- JWT signed with HS256 — secret from env var JWT_SECRET — expiry 7 days
- JWT payload: { userId, email, iat, exp } — nothing else
- Email verification: crypto.randomUUID() → saved to user.emailVerifyToken → link sent → on click set emailVerified: true, clear token
- Password reset: UUID → passwordResetToken + passwordResetExp (now + 1hr) → link sent → validate token + expiry → hash new password → clear fields
- Account lockout: loginAttempts >= 5 → lockUntil = Date.now() + 15 min → reject with 423 Locked
- On successful login: reset loginAttempts = 0, clear lockUntil

6.2 Google OAuth (GA4 + GSC + Google Ads)

- Strategy: passport-google-oauth20
- Scopes: https://www.googleapis.com/auth/analytics.readonly https://www.googleapis.com/auth/webmasters.readonly https://www.googleapis.com/auth/adwords openid email profile
- accessType: 'offline' — required to receive refresh_token
- prompt: 'consent' — required to get refresh_token on every auth, not just first time

- On callback: find or create User by googleId → upsert GoogleToken with encrypted tokens → sign JWT → redirect to /auth/callback?token=JWT
- All three Google services (GA4, GSC, Google Ads) share the same GoogleToken — one OAuth connection covers all three

6.3 Facebook OAuth (Meta Ads)

- Strategy: passport-facebook with profileFields: ['id', 'displayName', 'email']
- Scopes: ads_read, ads_management, business_management, email, public_profile
- Exchange short-lived token for long-lived token immediately after OAuth callback using Graph API
- Long-lived token endpoint: GET /oauth/access_token?grant_type=fb_exchange_token&client_id=&client_secret=&fb_exchange_token=
- Save encrypted long-lived token + expiresAt to FacebookToken document
- On callback: find or create User by facebookId → upsert FacebookToken → sign JWT → redirect to /auth/callback?token=JWT

6.4 Token Encryption — /server/utils/encrypt.js

- Algorithm: AES-256-GCM
- Key: 32-byte Buffer from env var ENCRYPTION_KEY (64 hex chars)
- Format stored in DB: iv:authTag:encryptedData — all as hex strings joined by colon
- Export: encrypt(plaintext) → string and decrypt(ciphertext) → string
- Apply to: GoogleToken.accessToken, GoogleToken.refreshToken, FacebookToken.accessToken

7. AI Prompt Engine — Full Specification

> ⚙️ **AGENT: Implement PromptBuilder as a class in /server/services/promptBuilder.js. Load all .txt prompt files at server startup using fs.readFileSync — not at request time. The PromptBuilder must handle partial data gracefully — if a user has not connected Facebook Ads, exclude that section from the context block without errors.**

7.1 System Prompt — /server/prompts/system.txt

Save this exact text to the file:

```
You are an expert digital marketing analyst embedded in a unified analytics dashboard.
You have access to data from up to four sources: Google Analytics 4, Google Search Console,
Google Ads, and Facebook Ads. Answer the user's question using only the data provided.
Rules:
1. Base every answer strictly on the provided data. Never invent metrics or figures.
2. If data for a source is not provided, do not reference that source in your answer.
3. Write for a non-technical audience. Avoid jargon. Explain acronyms on first use.
4. Format responses using markdown: headers, bullet points, bold for key numbers.
5. Keep responses under 400 words unless the user explicitly requests more detail.
6. Always end with one clear, actionable recommendation.
```

```
7. Never make predictions or performance guarantees.
8. Your response is advisory only — not a definitive statement of fact.
```

7.2 Context Block Format

The PromptBuilder.buildContext() method must produce this structure. Omit any section whose data was not provided:

```
## Analytics Context
Date Range: {startDate} to {endDate}

### Google Analytics 4
Users: {value} ({change}% vs prior period)
Sessions: {value} ({change}% vs prior period)
Bounce Rate: {value}%  |  Avg Session: {value}s  |  Page Views: {value}
Top Traffic Sources: {source}: {sessions} sessions, ...
Top Pages: {path}: {views} views, ...

### Google Search Console
Clicks: {value} ({change}%)  |  Impressions: {value} ({change}%)
Avg CTR: {value}%  |  Avg Position: {value}
Top Queries: {query}: {clicks} clicks, {ctr}% CTR, ...

### Google Ads
Total Spend: {currency}{value} ({change}%)
Impressions: {value}  |  Clicks: {value}  |  CTR: {value}%  |  Avg CPC: {currency}{value}
Conversions: {value}  |  ROAS: {value}
Top Campaigns by Spend: {name}: {currency}{spend}, {clicks} clicks, ...

### Facebook Ads
Total Spend: {currency}{value} ({change}%)
Reach: {value}  |  Impressions: {value}  |  Clicks: {value}  |  CTR: {value}%
CPM: {currency}{value}  |  CPC: {currency}{value}  |  ROAS: {value}
Top Campaigns by Spend: {name}: {currency}{spend}, {reach} reach, ...
```

7.3 Claude API Configuration — /server/services/claudeService.js

Parameter	Value	Notes
Model	claude-3-5-sonnet-20241022	Best balance of speed and intelligence for analytics Q&A
max_tokens	1024	Sufficient for concise cross-channel insights
Temperature	Not set	Claude does not support temperature param — omit entirely
Timeout	30 seconds	Reject and show error if no response within 30s
Retry	2 retries on 5xx errors only	Never retry on 4xx — those are configuration errors

Parameter	Value	Notes
Fallback	OpenAI GPT-4o via openaiService.js	If Claude returns non-2xx, try OpenAI with same prompt
Error msg	'AI service temporarily unavailable'	Return HTTP 503 if both Claude and OpenAI fail

8. React Frontend Specification

8.1 Routing Structure

⚙️ **AGENT: Use React Router v6. ProtectedRoute checks Zustand authStore for valid JWT. If no token → redirect to /login. If logged in but no accounts connected → redirect to /connect-accounts.**

Route	Component	Protected	Notes
/	Redirect	No	To /dashboard if authenticated, else /login
/login	LoginPage	No	Email/password + Google OAuth + Facebook OAuth buttons
/register	RegisterPage	No	Registration form
/verify-email/:token	VerifyEmailPage	No	Calls API, shows success/error
/forgot-password	ForgotPasswordPage	No	Email input form
/reset-password	ResetPasswordPage	No	New password form
/auth/callback	AuthCallbackPage	No	Reads ?token= from URL → saves to Zustand → replaces URL → redirects
/connect-accounts	ConnectAccountsPage	Yes	Connect Google (GA4+GSC+Ads) and Facebook. Show status for each.
/select-accounts	SelectAccountsPage	Yes	Pick specific GA4 property, GSC site, Ads account, FB ad account
/dashboard	DashboardPage	Yes	Unified dashboard with tabs for each data source + AI panel
/settings	SettingsPage	Yes	Account settings, disconnect integrations, delete account

8.2 Dashboard Layout

- Tab navigation for data sources: Overview | GA4 | Search Console | Google Ads | Facebook Ads
- Overview tab: combined KPIs from all connected sources side by side
- Each source tab: full metrics, charts, and tables for that source only
- Right panel (35% desktop width): AI weekly insight card + AI chat panel
- Header: app logo, source connection status badges, date range picker, user avatar

- All data areas show Tailwind animate-pulse skeleton loaders while fetching
- Disconnected source tabs show a 'Connect [Source]' prompt card instead of data

8.3 KPI Cards — All Sources

Card	Source	Format	% Change
Total Users	GA4	12,450	Yes — vs prior period
Sessions	GA4	18,230	Yes
Bounce Rate	GA4	42.3%	Yes — decrease is good (green)
Avg Session	GA4	2m 34s	No
Clicks	GSC	3,210	Yes
Impressions	GSC	84,500	Yes
Avg CTR	GSC	3.8%	Yes — increase is good (green)
Avg Position	GSC	14.2	Yes — decrease is good (green)
Ad Spend	Google Ads	$1,240.50	Yes — context dependent
Ad Clicks	Google Ads	4,820	Yes
Ad CTR	Google Ads	5.2%	Yes
ROAS	Google Ads	3.4x	Yes — increase is good
FB Spend	Facebook Ads	$980.00	Yes
FB Reach	Facebook Ads	42,100	Yes
FB CTR	Facebook Ads	2.1%	Yes
FB ROAS	Facebook Ads	2.8x	Yes — increase is good

8.4 Zustand Store Shapes

```
// authStore.js
{ token, user: { id, name, email, avatar }, setAuth(token,user), clearAuth() }
```

```
// accountsStore.js
{ ga4: { propertyId, propertyName }, gsc: { siteUrl },
  googleAds: { customerId, accountName, currencyCode },
  facebook: { adAccountId, adAccountName, currency },
  connectedSources: ['ga4','gsc','google-ads','facebook-ads'],
  setAccounts(accounts), clearAccounts() }
```

```
// dateRangeStore.js
{ preset: '7d'|'28d'|'30d'|'90d'|'custom',
  startDate: 'YYYY-MM-DD', endDate: 'YYYY-MM-DD',
  setPreset(preset), setCustomRange(start,end) }
```

9. Environment Variables

⚙️ **AGENT: Create .env.example at root with every variable below. Add .env to .gitignore. Server reads process.env. Client reads import.meta.env — only VITE_ prefixed vars are available client-side.**

Variable	Used By	Description	Example
NODE_ENV	Server	Environment flag	development
PORT	Server	Express listen port	5000
MONGODB_URI	Server	MongoDB Atlas connection string	mongodb+srv://user:pass@cluster.mongodb.net/analytics_platform
JWT_SECRET	Server	JWT signing secret — minimum 32 chars	your-long-random-secret-here
JWT_EXPIRES_IN	Server	JWT expiry duration	7d
ENCRYPTION_KEY	Server	AES-256 key — must be exactly 64 hex characters	a1b2c3d4... (64 hex chars)
GOOGLE_CLIENT_ID	Server	OAuth 2.0 Client ID from Google Cloud Console	123456.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET	Server	OAuth 2.0 Client Secret	GOCSPX-xxxx
GOOGLE_CALLBACK_URL	Server	Full Google OAuth callback URL	http://localhost:5000/api/auth/google/callback
GOOGLE_ADS_DEVELOPER_TOKEN	Server	Google Ads API developer token from Google Ads Manager	xxxx-xxxx-xxxx
FACEBOOK_APP_ID	Server	Facebook App ID from Meta Developer Console	1234567890
FACEBOOK_APP_SECRET	Server	Facebook App Secret	abcdef1234567890
FACEBOOK_CALLBACK_URL	Server	Full Facebook OAuth callback URL	http://localhost:5000/api/auth/facebook/callback
ANTHROPIC_API_KEY	Server	Anthropic Claude API key	sk-ant-xxxx
OPENAI_API_KEY	Server	OpenAI fallback API key	sk-xxxx
RESEND_API_KEY	Server	Resend.com transactional email key	re_xxxx
EMAIL_FROM	Server	Sender address for all emails	noreply@yourdomain.com
CLIENT_URL	Server	Frontend URL — used in CORS and email links	http://localhost:5173
VITE_API_BASE_URL	Client	Backend API base URL used by Axios	http://localhost:5000/api
VITE_GOOGLE_CLIENT_ID	Client	Same as GOOGLE_CLIENT_ID — for OAuth UI hint	123456.apps.googleusercontent.com
VITE_FACEBOOK_APP_ID	Client	Same as FACEBOOK_APP_ID —	1234567890

Variable	Used By	Description	Example
		for FB SDK if needed	

10. Required NPM Packages

10.1 Server — /server/package.json

Package	Version	Purpose
express	^4.18	HTTP server and routing
mongoose	^8.x	MongoDB ODM
passport	^0.7	Auth middleware
passport-google-oauth20	^2.0	Google OAuth 2.0 strategy
passport-facebook	^3.0	Facebook OAuth 2.0 strategy
jsonwebtoken	^9.x	JWT sign and verify
bcryptjs	^2.4	Password hashing
googleapis	^140.x	GA4 + GSC official API client
google-ads-api	^16.x	Google Ads API client (GAQL)
facebook-nodejs-business-sdk	^20.x	Official Meta Marketing API SDK
@anthropic-ai/sdk	^0.30	Anthropic Claude API client
openai	^4.x	OpenAI GPT-4o fallback client
cors	^2.8	CORS middleware
helmet	^7.x	Security HTTP headers
express-rate-limit	^7.x	Rate limiting per IP and per user
dotenv	^16.x	Load .env files
morgan	^1.10	HTTP request logging
node-cache	^5.x	In-memory API response cache
node-cron	^3.x	Scheduled jobs (Facebook token refresh reminders)
resend	^3.x	Transactional email
uuid	^9.x	Random token generation
nodemon	^3.x	Dev auto-restart (devDependency)

10.2 Client — /client/package.json

Package	Version	Purpose
react	^18.x	UI framework
react-dom	^18.x	React DOM renderer
react-router-dom	^6.x	Client-side routing
axios	^1.x	HTTP client with interceptors
zustand	^4.x	Lightweight global state
recharts	^2.x	Chart components (Line, Bar, Pie, Area)
react-markdown	^9.x	Render AI markdown responses
remark-gfm	^4.x	GitHub Flavored Markdown plugin
date-fns	^3.x	Date formatting and calculations
react-hot-toast	^2.x	Toast notifications
@heroicons/react	^2.x	SVG icon set compatible with Tailwind
vite	^5.x	Build tool (devDep)
@vitejs/plugin-react	^4.x	Vite React plugin (devDep)
tailwindcss	^3.x	Utility CSS (devDep)
autoprefixer	^10.x	PostCSS autoprefixer (devDep)
postcss	^8.x	PostCSS (devDep)

11. Error Handling & Edge Cases

> ⚙️ **AGENT: Wrap all async Express route handlers with catchAsync.js utility. Zero unhandled promise rejections allowed. All 4 API integrations must fail independently — if Facebook Ads API fails, GA4, GSC, and Google Ads must still load normally.**

Scenario	Server Behaviour	Client Behaviour
Google token expired	Auto-refresh before every Google API call. If refresh fails → 401 GOOGLE_AUTH_EXPIRED	Axios interceptor catches 401 code → redirect to /auth/google
Facebook token expired/revoked	Check expiresAt before every FB call. If expired → 401 FACEBOOK_AUTH_EXPIRED	Redirect to /connect-accounts with message to reconnect Facebook
Google Ads no access	Return 403 with message: 'No Google Ads accounts found for this Google account'	Show connect prompt card in Google Ads tab
Facebook no ad accounts	Return 200 with empty array []	Show: 'No Facebook Ad Accounts found for this account'

Scenario	Server Behaviour	Client Behaviour
Any API 429 rate limit	Exponential backoff: 1s→2s→4s→8s (4 retries). Cache partial results. Return 503 if all fail	Show: 'Data is loading, please wait...' — retry silently
Any API 500	Log full error server-side. Return 503 with generic message — never expose raw API error	Show error card per source with Retry button
Claude timeout (>30s)	Cancel, try OpenAI GPT-4o. If both fail → 503	Show: 'AI is taking too long. Please try again.'
Source not connected	Return 400 with code: SOURCE_NOT_CONNECTED	Show connect card for that source — do not crash the page
Invalid date range	Validate: start <= end, no future end date, max 2 years range → 400	Disable submit, show inline validation message
JWT expired	Return 401 on all protected routes	Axios interceptor → clear Zustand → redirect to /login
Account deletion	Cascade delete: User + GoogleToken + FacebookToken + UserAccounts + all cache + all AI logs	Redirect to /login with toast: 'Account deleted'

12. Security Requirements

Requirement	Implementation
Security headers	helmet() middleware — sets CSP, HSTS, X-Frame-Options, X-Content-Type-Options automatically
CORS	Allow only CLIENT_URL env var origin. credentials: true. Methods: GET, POST, PUT, DELETE, OPTIONS
Rate limits	Global: 100 req/15min per IP. Auth: 10 req/15min per IP. AI: 20 req/hour per user
Input validation	express-validator on all POST/PUT routes. Reject requests with unexpected or malformed fields
NoSQL injection	Never pass raw req.body directly to Mongoose. Always destructure expected fields explicitly
XSS	React escapes by default. AI markdown via react-markdown only — never dangerouslySetInnerHTML
Token storage	All Google and Facebook tokens encrypted with AES-256-GCM before saving to MongoDB
JWT storage	JWT in Zustand (memory) + localStorage. Cleared on logout. Never in cookies for MVP
Secrets	All secrets in .env — never in source code. .env in .gitignore. .env.example for docs

Requirement	Implementation
Facebook secret	FACEBOOK_APP_SECRET never sent to client. All FB API calls via backend only
HTTPS	Enforced by hosting platform (Vercel/Railway force HTTPS). Redirect HTTP to HTTPS
Dependency audit	Run npm audit before every production deployment. Fix all high severity before launch

13. Non-Functional Requirements

Category	Metric	Target
Performance	Dashboard initial load (TTI)	< 3 seconds on broadband
Performance	Cached API response	< 500ms per source
Performance	Uncached external API call	< 10 seconds (Facebook is slower than Google)
Performance	AI response time	< 15 seconds
Performance	Vite production bundle	< 600KB gzipped — use React.lazy() for route splitting
Reliability	API independence	Each of the 4 sources fails independently — no cascade failures
Reliability	AI fallback	OpenAI GPT-4o if Claude fails — user never sees 'AI unavailable' unless both fail
Scalability	Stateless API	No local filesystem state — any server instance can serve any request
UX	Loading states	Skeleton loaders for every async data section — never blank
UX	Disconnected sources	Show connect prompt card for any source not yet connected
UX	Mobile responsive	Usable on screens >= 375px. Tailwind responsive prefixes throughout
GDPR	Visitor data	Zero personal visitor data stored — aggregated metrics only
GDPR	Data retention	AI messages: 90 days. Cache: 30 min. Insight: 24hr. Tokens: until disconnected
GDPR	Right to deletion	Full cascade delete of all user data within 24 hours of request

14. Vite & Tailwind Configuration

14.1 vite.config.js

```
import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: { '/api': { target: 'http://localhost:5000', changeOrigin: true } }
  }
})
```

14.2 tailwind.config.js

```
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: { extend: { colors: {
    primary: { 50:'#eff6ff', 500:'#3b82f6', 600:'#2563eb', 700:'#1d4ed8' },
    surface: { DEFAULT:'#ffffff', muted:'#f8fafc' }
  }}},
  plugins: []
}
```

14.3 index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

⚙️ **AGENT: No custom CSS beyond these three lines. All styling via Tailwind utility classes in JSX className props.**

14.4 Root package.json — Dev Script

```
"scripts": {
  "dev": "concurrently \"npm run dev:server\" \"npm run dev:client\"",
  "dev:server": "cd server && npm run dev",
  "dev:client": "cd client && npm run dev",
  "install:all": "npm install && cd server && npm install && cd ../client && npm install"
}
```

15. Hosting & Deployment

15.1 Database — MongoDB Atlas (Confirmed)

Step	Action	Detail
1	Create Atlas account	cloud.mongodb.com — sign up with Google or email
2	Create cluster	M0 Free for dev. Upgrade to M10 (~$57/mo) before production launch
3	Create DB user	Database Access → Add New User → readWriteAnyDatabase role

Step	Action	Detail
4	Whitelist network	Network Access → 0.0.0.0/0 for production or restrict to hosting server IP
5	Get connection string	Clusters → Connect → Drivers → copy URI → replace <password>
6	Set MONGODB_URI in .env	mongodb+srv://user:pass@cluster.mongodb.net/analytics_platform?retryWrites=true&w=majority
7	Enable TTL indexes	Both AnalyticsCache and WeeklyInsight use TTL indexes — these auto-activate on first insert

15.2 Frontend — Vercel (Recommended) or Netlify

Config	Vercel	Netlify
Build command	cd client && npm run build	cd client && npm run build
Output dir	client/dist	client/dist
Config file	client/vercel.json	client/netlify.toml
Env vars	Project → Settings → Env Vars	Site → Environment Variables
Redirect rule	{ rewrites: [{ source:'/(.*)', destination:'/index.html' }] }	[[redirects]] from = '/*' to = '/index.html' status = 200
Free bandwidth	100 GB/month	100 GB/month
Verdict	Recommended	Good alternative

⚙️ AGENT: The redirect/rewrite rule is mandatory for React Router. Without it, refreshing any page other than / returns 404. Create both vercel.json and netlify.toml files so the team can deploy to either without code changes.

15.3 Backend — Railway (Recommended) or Render

Config	Railway	Render
Start command	node server.js	node server.js
Node version	engines in package.json: >=18.0.0	RENDER_NODE_VERSION=18 in env vars
Env vars	Service → Variables tab	Service → Environment tab
Health check	GET /health → 200 { status: 'ok' }	Settings → Health Check Path: /health
Free tier	$5 credit — no sleep on paid	Sleeps after 15min — use Starter $7/mo for production
Cost (paid)	~$5-10/month	~$7/month (Starter)
Verdict	Recommended	Good alternative

⚙️ AGENT: Add GET /health endpoint to app.js: app.get('/health', (req, res) => res.status(200).json({ status: 'ok', timestamp: new Date() })). This is required for both Railway and Render health checks.

15.4 Google Cloud Console Setup for OAuth + Google Ads

Step	Action	Detail
1	Create project	console.cloud.google.com → New Project
2	Enable APIs	Enable: Google Analytics Data API, Search Console API, Google Ads API
3	Create OAuth credentials	APIs & Services → Credentials → Create OAuth Client ID → Web Application
4	Add redirect URIs	Dev: http://localhost:5000/api/auth/google/callback | Prod: https://your-backend.railway.app/api/auth/google/callback
5	Get Client ID + Secret	Copy GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET to .env
6	Google Ads developer token	Google Ads Manager account → Tools → API Center → Apply for developer token
7	Add test users (dev)	OAuth Consent Screen → Test Users → add emails that will test the app
8	Publish for production	OAuth Consent Screen → Publish App — required for real users to connect

15.5 Meta Developer Console Setup for Facebook Ads

Step	Action	Detail
1	Create Meta App	developers.facebook.com → My Apps → Create App → Business type
2	Add Marketing API	App Dashboard → Add Product → Marketing API
3	Get App ID + Secret	Settings → Basic → copy App ID and App Secret to .env
4	Add Facebook Login	App Dashboard → Add Product → Facebook Login → Web
5	Set Valid OAuth URIs	Facebook Login → Settings → Valid OAuth Redirect URIs → add callback URL
6	Request permissions	App Review → Permissions → request: ads_read, ads_management, business_management
7	Dev mode	In development: only test users can authenticate. Add test users in Roles → Test Users
8	Go Live for production	App Dashboard → toggle to Live mode — required for real users to connect their ad accounts

15.6 Production Deployment Checklist

#	Item	Pass Condition
1	MongoDB Atlas M10 running	Atlas dashboard shows active cluster at M10 tier
2	All server env vars set on hosting	Railway/Render vars match every entry in .env.example
3	All client env vars set on hosting	Vercel/Netlify vars include VITE_API_BASE_URL pointing to production backend

#	Item	Pass Condition
4	CORS CLIENT_URL matches frontend URL	No CORS errors in browser console on first request
5	Google OAuth redirect URI updated	Google Cloud Console → Credentials → production callback URL added
6	Facebook OAuth redirect URI updated	Meta Developer Console → Facebook Login → production callback URL added
7	GET /health returns 200	curl https://your-backend-url/health returns { status: 'ok' }
8	Google Ads developer token approved	google-ads-api calls return data — not 'developer token not approved'
9	Facebook App is Live	Non-test users can authenticate with Facebook
10	Frontend loads without console errors	Browser DevTools → Console → zero errors on load
11	All 4 OAuth flows work in production	End-to-end test: connect Google, connect Facebook, select all accounts
12	Dashboard shows real data from all sources	All connected KPI cards show actual data — not zeros or errors
13	AI question returns response	Type a question — AI responds using data from all connected sources
14	HTTPS enforced	http:// redirects to https:// automatically
15	No secrets in GitHub	Search entire repo for API keys — zero results

16. Agent Build Acceptance Checklist

⚙️ **AGENT: The build is COMPLETE only when every single item below passes. Do not mark done until all 25 pass.**

#	Check Item	Pass Condition
1	Folder structure	Exact structure from Section 4 exists — all files and folders present
2	MongoDB connected	Server starts, mongoose.connect() succeeds, no DB errors in logs
3	All 8 Mongoose models	All models from Section 3 exist with correct fields, types, and TTL indexes
4	Registration	POST /api/auth/register creates user + sends verification email
5	Email verification	GET /api/auth/verify-email/:token sets emailVerified: true
6	Email/password login	Returns JWT. Wrong password → 401. Locked account → 423
7	Google OAuth	Full flow: /auth/google → consent → callback → JWT → redirect to client
8	Facebook OAuth	Full flow: /auth/facebook → consent → callback → JWT → redirect to client
9	Token encryption	GoogleToken and FacebookToken documents in Atlas contain encrypted strings — not plain tokens

#	Check Item	Pass Condition
10	Google token auto-refresh	Expired Google access token is refreshed silently before GA4/GSC/Ads API call
11	Facebook token refresh	node-cron job runs daily. Sends reminder email 7 days before FB token expires
12	Account selection	POST /api/accounts/select saves all 4 source accounts to UserAccounts
13	GA4 data	All 7 /api/ga4/* endpoints return real data from Google Analytics 4
14	GSC data	All 6 /api/gsc/* endpoints return real data from Search Console
15	Google Ads data	All 6 /api/google-ads/* endpoints return real campaign and spend data
16	Facebook Ads data	All 6 /api/facebook-ads/* endpoints return real campaign and spend data
17	API caching	Second identical request returns from cache in < 500ms. TTL expires after 30 min
18	Source independence	If one API fails, other 3 sources load normally — no cascade failure
19	AI ask endpoint	POST /api/ai/ask returns markdown. Conversation saved to MongoDB
20	Weekly insight	GET /api/ai/weekly-insight returns AI markdown covering all connected sources
21	React dashboard renders	All tabs, KPI cards, charts, tables render with real data from all 4 sources
22	Disconnected source UX	Unconnected sources show a connect prompt card — not an error
23	AI chat UI	User asks question → AI response renders as formatted markdown in chat panel
24	Account deletion	DELETE /api/auth/me removes user + all related docs from MongoDB
25	Security headers	curl -I on production backend shows: X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security
26	Super Admin tab visible	User with SUPER_ADMIN_EMAIL env var logs in → Admin tab appears in dashboard nav
27	API keys load in admin	Admin tab shows all 4 integration config forms with current masked values
28	Save API key works	Admin saves a key → POST /api/admin/config → value encrypted in MongoDB → services use new value immediately
29	Non-admin cannot access	Any user other than super admin gets 403 on all /api/admin/* routes. Admin tab is not rendered in their UI
30	Key masking works	GET /api/admin/config returns masked values only e.g. sk-ant-****1234 — never full keys

17. Super Admin Panel

The Super Admin panel allows the platform owner to manage all API keys and integration credentials directly from the dashboard UI — no server access, no .env file editing required after initial deployment. This is the only feature of the Super Admin in Phase 1.

> ⚙️ **AGENT: The Super Admin is identified solely by matching the logged-in user's email against the SUPER_ADMIN_EMAIL environment variable. There is no role field in the User schema. The admin tab is conditionally rendered in React only when authStore.user.email === import.meta.env.VITE_SUPER_ADMIN_EMAIL. All /api/admin/* routes are protected by a superAdminOnly middleware that performs the same email check server-side.**

17.1 Super Admin Access Control

Layer	Implementation	Detail
Env var	SUPER_ADMIN_EMAIL in .env	The single email address that has super admin access. Set once at deployment — never changes without a redeploy
Server middleware	superAdminOnly.js in /server/middleware/	Checks req.user.email === process.env.SUPER_ADMIN_EMAIL. Returns 403 { message: 'Access denied' } for any other user. Applied to ALL /api/admin/* routes
Client guard	AdminRoute component in React	Wraps /dashboard/admin route. Checks authStore.user.email === import.meta.env.VITE_SUPER_ADMIN_EMAIL. Renders 'Access Denied' page for non-admins
Nav tab	Conditional in DashboardLayout.jsx	Admin tab in nav only rendered when user email matches VITE_SUPER_ADMIN_EMAIL. Invisible to all regular users
Double protection	Both client guard + server middleware required	Client guard prevents UI access. Server middleware prevents direct API access. Both must be in place

17.2 PlatformConfig Schema — /server/models/PlatformConfig.js

Stores all platform-level API credentials in MongoDB. One document per config key. All values AES-256-GCM encrypted before saving.

Field	Type	Required	Notes
_id	ObjectId	auto	MongoDB default
key	String	Yes	Unique identifier. Enum — see Section 17.3 for all valid keys
value	String	Yes	AES-256-GCM encrypted value. Decrypted only at runtime in service files
label	String	Yes	Human-readable label shown in admin UI e.g. 'Google Client ID'

Field	Type	Required	Notes
group	String	Yes	Enum: google | facebook | anthropic | openai — groups keys in UI
isSecret	Boolean	Yes	Default true. If true, value is masked in GET responses
updatedBy	String	Yes	Email of admin who last updated this key
updatedAt	Date	auto	Mongoose timestamps
createdAt	Date	auto	Mongoose timestamps

⚙️ **AGENT: On first server startup, if PlatformConfig collection is empty, seed it with all keys from Section 17.3 using values read from process.env. This means the app works immediately after first deployment using .env values, and the admin can update them later from the UI without touching the server.**

17.3 Config Keys — All Valid Keys

Key (stored in DB)	Label (shown in UI)	Group	Used By
GOOGLE_CLIENT_ID	Google OAuth Client ID	google	GA4, GSC, Google Ads OAuth flow
GOOGLE_CLIENT_SECRET	Google OAuth Client Secret	google	GA4, GSC, Google Ads OAuth flow
GOOGLE_ADS_DEVELOPER_TOKEN	Google Ads Developer Token	google	Google Ads API — GAQL queries
FACEBOOK_APP_ID	Facebook App ID	facebook	Facebook OAuth + Meta Marketing API
FACEBOOK_APP_SECRET	Facebook App Secret	facebook	Facebook OAuth token exchange
ANTHROPIC_API_KEY	Anthropic Claude API Key	anthropic	Primary AI model — Claude 3.5 Sonnet
OPENAI_API_KEY	OpenAI API Key (Fallback)	openai	Fallback AI model — GPT-4o
RESEND_API_KEY	Resend Email API Key	other	Transactional email delivery

17.4 Admin API Endpoints — /api/admin

⚙️ **AGENT: All routes in /api/admin/* must be protected by both the standard JWT auth middleware AND the superAdminOnly middleware — in that order. Never expose decrypted values in any response.**

Method	Endpoint	Description	Request Body	Response
GET	/config	Get all config keys with masked values	—	200 [{ key, label, group, maskedValue, updatedAt }]
GET	/config/:key	Get single config key with masked value	—	200 { key, label, group, maskedValue, updatedAt, updatedBy }
POST	/config	Create or update a	{ key, value }	200 { message: 'Config saved',

Method	Endpoint	Description	Request Body	Response
		config key value		key, updatedAt }
POST	/config/bulk	Save multiple config keys at once	{ configs: [{key,value}] }	200 { message: 'X configs saved', results: [{key, success}] }
POST	/config/test/:key	Test if a saved API key works	—	200 { success: bool, message: string }

17.5 Value Masking Logic

⚙️ **AGENT: Implement maskValue(str) in /server/utils/maskValue.js. Apply to all GET /api/admin/config responses. Never return raw decrypted values to the client under any circumstance.**

Key Type	Masking Rule	Example Output
Anthropic API key	Show prefix sk-ant- then **** then last 4 chars	sk-ant-****a1b2
OpenAI API key	Show prefix sk- then **** then last 4 chars	sk-****x9y8
Google Client ID	Show first 6 chars then **** then last 4 chars	123456****7890.apps.googleusercontent.com
Google Client Secret	Show first 4 chars then **** only	GOCS****
Google Ads Token	Show first 4 chars then **** then last 4 chars	xxxx****yyyy
Facebook App ID	Show full value — not a secret (public)	1234567890 (unmasked)
Facebook App Secret	Show first 4 chars then **** only	abcd****
Resend API key	Show prefix re_ then **** then last 4 chars	re_****z1z2

17.6 API Key Test Endpoint Logic

POST /api/admin/config/test/:key allows the admin to verify a saved API key works without leaving the admin panel. The server decrypts the saved value and makes a minimal test call to the respective service:

Key	Test Action	Pass Condition
GOOGLE_CLIENT_ID / SECRET	Attempt to exchange a dummy code — expect error 400, not 401/403	Response indicates credentials are valid format
GOOGLE_ADS_DEVELOPER_TOKEN	Call Google Ads API list accessible customers with stored Google token	Returns customer list or empty array — not 401

Key	Test Action	Pass Condition
FACEBOOK_APP_ID / SECRET	Call GET /debug_token on Graph API to verify app credentials	Returns valid token debug info — not OAuthException
ANTHROPIC_API_KEY	Send minimal message: 'Say OK' with max_tokens: 5	Returns 200 with content — not 401/403
OPENAI_API_KEY	Send minimal chat completion: 'Say OK' with max_tokens: 5	Returns 200 with choices — not 401/403
RESEND_API_KEY	Call GET https://api.resend.com/domains to list domains	Returns 200 — not 401

17.7 Admin Tab UI — React

⚙️ **AGENT: Create AdminPage.jsx in /client/src/pages/. Add /dashboard/admin route wrapped in AdminRoute. Add Admin tab to dashboard nav — visible only when user email matches VITE_SUPER_ADMIN_EMAIL.**

- Page title: Platform API Configuration
- Layout: 4 collapsible sections, one per group — Google, Facebook, Anthropic / OpenAI, Other
- Each section has a group icon and a green 'Configured' / red 'Not Set' badge
- Each config key shows: label, masked current value, a text input to enter new value, a Save button, and a Test button
- Input type: password field (type='password') so value is hidden while typing — toggle show/hide button
- On Save: POST /api/admin/config with { key, value } → show success toast on 200 → refresh masked value displayed
- On Test: POST /api/admin/config/test/:key → show green 'Connection successful' or red 'Test failed: {message}'
- Bulk Save button at bottom of each group section — saves all fields in that group in one POST /api/admin/config/bulk call
- Show last updated timestamp and 'Updated by: {email}' under each saved field
- Warning banner at top of page: 'Changes take effect immediately. Existing user sessions use the new credentials on their next API call.'

17.8 Runtime Config Loading

⚙️ **AGENT: All 4 service files (ga4Service.js, gscService.js, googleAdsService.js, facebookAdsService.js, claudeService.js, openaiService.js) must load their API credentials from MongoDB PlatformConfig at runtime — NOT from process.env directly. Create a configService.js that fetches, decrypts, and caches config values in memory with a 5-minute TTL. Services call configService.get('ANTHROPIC_API_KEY') instead of process.env.ANTHROPIC_API_KEY.**

File	Change Required
/server/services/configService.js	NEW FILE — fetches all PlatformConfig docs, decrypts values, caches in-memory with 5-min TTL. Exports get(key) method
/server/services/claudeService.js	Replace process.env.ANTHROPIC_API_KEY with await configService.get('ANTHROPIC_API_KEY')

File	Change Required
/server/services/openaiService.js	Replace process.env.OPENAI_API_KEY with await configService.get('OPENAI_API_KEY')
/server/services/ga4Service.js	Replace process.env.GOOGLE_CLIENT_ID/SECRET with configService.get() equivalents
/server/services/ googleAdsService.js	Replace process.env.GOOGLE_ADS_DEVELOPER_TOKEN with await configService.get('GOOGLE_ADS_DEVELOPER_TOKEN')
/server/services/ facebookAdsService.js	Replace process.env.FACEBOOK_APP_ID/SECRET with configService.get() equivalents
/server/config/passport.js	Passport strategies must also call configService.get() for Google and Facebook credentials at strategy init time

17.9 New Env Vars for Super Admin

Variable	Used By	Description	Example
SUPER_ADMIN_EMAIL	Server + Client	Email address of the super admin — compared against logged-in user email	admin@yourcompany.com
VITE_SUPER_ADMIN_EMAIL	Client only	Same value as SUPER_ADMIN_EMAIL — needed for React conditional rendering	admin@yourcompany.com

⚙️ **AGENT: These are the ONLY two new env vars. All other API credentials (GOOGLE_CLIENT_ID, ANTHROPIC_API_KEY, etc.) are still needed in .env for the initial seed on first deployment. After that, the admin can update them from the UI and they are stored in MongoDB PlatformConfig.**

17.10 New Files Summary for Super Admin

File Path	Type	Purpose
/server/models/PlatformConfig.js	New Model	MongoDB schema for storing encrypted API config keys
/server/services/configService.js	New Service	Fetches + decrypts + caches PlatformConfig values. Used by all API service files
/server/middleware/ superAdminOnly.js	New Middleware	Checks req.user.email === process.env.SUPER_ADMIN_EMAIL. Returns 403 if not matched
/server/controllers/ adminController.js	New Controller	Handles all /api/admin/* route logic
/server/routes/adminRoutes.js	New Routes	Express router for /api/admin/* — applies auth + superAdminOnly middleware
/server/utils/maskValue.js	New Utility	maskValue(str, key) function — returns masked string based on key type

File Path	Type	Purpose
/client/src/pages/AdminPage.jsx	New Page	Super admin UI — 4 grouped sections with masked fields, save, and test buttons
/client/src/components/ui/AdminRoute.jsx	New Component	Route wrapper — renders children only if user email matches VITE_SUPER_ADMIN_EMAIL
/client/src/api/adminApi.js	New API file	Axios calls for GET /config, POST /config, POST /config/bulk, POST /config/test/:key

— End of Document —

Version 1.0 | Final | MERN Stack | GA4 · GSC · Google Ads · Facebook Ads | Super Admin

Authored with Claude.ai | Built for Cursor Agentic Development